RESILIENCE PRIMER · AZURE + DATABRICKS

High availability, backup and disaster recovery: staying up, and getting back

Three different tools for three different failures. **High availability** rides through a zone failure. **Backup** undoes bad data. **Disaster recovery** moves the platform to another region. None of them replaces the others.

> LEARNING OBJECTIVE
>
> Describe high availability, backup and disaster recovery; compare managed and self-built DR; define RTO, RPO and replication approaches; and outline DR runbooks and deploying a secondary region as code.

01 Why: failures come in different sizes

Every data platform will eventually lose a machine, a zone, a table or a whole region. The question for each workload is how long the business can wait, and how much recent data it can afford to lose. The answer decides which protection you need and how much you spend on it.

WHAT FAILS	HIGH AVAILABILITY	BACKUP	DISASTER RECOVERY
A cluster node or VM fails	✓ Cluster manager replaces the node from another zone	– Not needed	– Not needed
An availability zone goes down	✓ Control plane recovers in about 15 minutes; ZRS storage keeps data	– Not needed	– Not needed
A bad job overwrites a table	× The bad write is highly available too	✓ RESTORE to an earlier Delta version	× Replicated to the secondary
Someone drops a table	× Dropped in every zone	✓ UNDROP within 7 days, or restore a Deep Clone	× The drop replicates too
Ransomware or malicious deletion	× No protection	✓ Immutable or soft-deleted backups the attacker can't reach	× Deletions replicate
The whole region is unavailable	× All zones are in the region	~ Only if backups are stored in another region	✓ Fail over to the secondary region

Disaster recovery does not fix bad data

Replication copies whatever the primary holds. A corrupted or deleted table is corrupted or deleted in both regions within minutes. Only backups and Delta history take you back in time.

High availability, backup and DR: what each one covers

IN ONE REGION · AUTOMATIC

High availability

Keeps running through the loss of a machine or one availability zone, usually without anyone acting.

Control plane
Core services are multi-zonal and recover from a zone failure in about 15 minutes, in regions with multiple zones.

Compute
Worker nodes come from several zones and are replaced if lost. Losing the driver restarts the cluster and job.

Storage
Choose ZRS or GZRS for ADLS and workspace storage so data survives a zone loss.

Jobs
Task retries and idempotent writes turn a restart into a short delay.

`Your job:` pick a multi-zone region and zone-redundant storage.

POINT IN TIME · UNDO

Backup

Takes data back to a moment before something went wrong. The only protection against corruption and deletion.

Delta time travel
Query or RESTORE earlier versions. History lasts only until VACUUM removes old files (7 days by default).

UNDROP
Recovers a dropped Unity Catalog managed table within 7 days.

Deep Clone
Scheduled full or incremental copy of a table to a separate backup location or region.

Storage protection
Soft delete, versioning and immutability policies on the backup storage account.

`Your job:` set retention, schedule backups and test restores.

SECOND REGION · FAILOVER

Disaster recovery

Moves the platform to another region when the primary region is lost or unusable.

Secondary workspace
Same account, same Entra ID identities, its own metastore and storage.

Replication
Data, metadata, code and configuration kept in step with the primary.

Failover
A decision, a runbook and a procedure that redirect users and jobs.

Region choice
Confirm every service and VM type you use is available in the secondary.

`Your job:` choose a strategy, build it, and rehearse it.

03 RTO, RPO and replication approaches

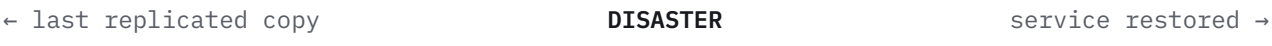
TWO NUMBERS EVERY WORKLOAD NEEDS

RPO · Recovery point objective

How much recent data you can afford to lose, measured back from the moment of failure to the last copy in the secondary. **"We can lose at most 15 minutes of orders."**

RTO · Recovery time objective

How long the business can wait for the service to work again, measured forward from the failure. **"Finance dashboards back within 4 hours."**



Databricks stores notebooks, jobs and metadata in its control plane, so its RPO covers changes to those. Your table data lives in your ADLS accounts; you set and meet the RPO for it.

Lower RPO and RTO cost more. Typical targets for each strategy:

Backup and restore Rebuild after the event

Nothing runs in the secondary region. Infrastructure is redeployed from code after the disaster, and data is restored from Deep Clone backups or geo-replicated storage.

TYPICAL RPO
Hours to 1 day

TYPICAL RTO
Hours to days

RELATIVE COST
Low

IN THE SECONDARY REGION

Backup storage only. Terraform and bundles ready to run.

GOOD FIT FOR

Internal reporting and development workspaces that can wait a day.

Figures are typical ranges for planning, not guarantees. Measure your own in DR tests.

Active-passive, self-built Warm copy, scheduled sync

A secondary workspace is deployed and kept in sync by CI/CD and scheduled Deep Clone jobs. Compute stays off until failover.

TYPICAL RPO
Sync interval, e.g. 1–4 h

TYPICAL RTO
1–4 hours

RELATIVE COST
Medium

IN THE SECONDARY REGION
Workspace, metastore and storage deployed; jobs paused; data refreshed on a schedule.

GOOD FIT FOR
Most production analytics with an RTO of a few hours.

Figures are typical ranges for planning, not guarantees. Measure your own in DR tests.

Active-passive, managed DR Continuous replication

Databricks continuously replicates catalogs, managed table data and workspace assets. Failover is a button in the account console and clients follow a stable URL.

TYPICAL RPO
Minutes (replication point)

TYPICAL RTO
Minutes to 1 hour

RELATIVE COST
Medium–high

IN THE SECONDARY REGION
Read-only replicas of in-scope catalogs and assets; compute unavailable until failover.

GOOD FIT FOR
Business-critical workloads that must be back within the hour.

Figures are typical ranges for planning, not guarantees. Measure your own in DR tests.

Active-active Both regions run all the time

Every job runs in both regions and is only complete when both succeed. Users can switch region at any time.

TYPICAL RPO

Near zero

TYPICAL RTO

Near zero to minutes

RELATIVE COST

High (about 2× compute)

IN THE SECONDARY REGION

A second full production deployment with strict CI/CD so neither side changes on its own.

GOOD FIT FOR

The few workloads where any downtime or data loss is unacceptable.

Figures are typical ranges for planning, not guarantees. Measure your own in DR tests.

REPLICATION APPROACHES · WHAT TO COPY, AND HOW

A region failover needs more than data. Every row below must exist in the secondary.

ASSET	APPROACH	WATCH OUT FOR
UC managed tables	Managed DR, or scheduled DEEP CLONE into the secondary catalog.	Clone runs define your RPO. Tables with row filters or column masks aren't replicated by managed DR.
External data and raw files	Azure Storage replication (GRS/GZRS, object replication) or copy jobs; convert to Delta and clone where possible.	Don't rely on GRS for the workspace root storage account.
Streaming checkpoints	Keep in your own storage and replicate alongside the data they describe.	Paths inside checkpoints may point at the primary; plan the restart point and de-duplicate.
Catalogs, schemas, grants	Managed DR, or Terraform applied to both metastores.	External locations and storage credentials must be created in the secondary yourself.
Notebooks, jobs, clusters	Managed DR asset replication, or Asset Bundles deployed to both workspaces.	Keep secondary jobs paused and pools at zero idle until failover.
Identities	Same Entra ID tenant; automatic identity management or SCIM at account level.	Nothing to copy if identities are account-level. Local workspace users break DR.
Secrets and keys	A Key Vault in each region, populated by your secret pipeline.	Managed DR doesn't replicate secrets. Some values differ by region.
Network and Azure resources	Terraform or Bicep with region variables.	Private endpoints, firewalls and DNS for the secondary must exist before the event.

04 Managed versus self-built disaster recovery

Both start from the same place: a second workspace and metastore in another region, with the same identities. The difference is who builds and runs the replication and the failover. Databricks recommends managed DR, and self-built DR for what it doesn't cover.

Managed disaster recovery

RECOMMENDED

Databricks runs the replication pipeline and the failover. You configure a **failover group** in the account console under Resilience.

> **Replicates:** Unity Catalog metadata, managed table data, views, functions and grants; optionally notebooks, jobs, warehouses, clusters, dashboards and files with their ACLs.

> **Stable URL:** one address that always points at the current primary, so UI, JDBC/ODBC and API clients keep working after failover.

> **Failover:** from the account console, in minutes. Replication then reverses automatically.

> **Monitoring:** the replication point shows how current the secondary is; system.replication.states tracks RPO and errors.

> **Requires:** Premium plan and the Mission Critical add-on on both workspaces, serverless compute, a secondary metastore. Access is gated through your account team.

> **Trade-off:** secondary catalogs are read-only and its compute is unavailable while replication runs.

Self-built (DIY) disaster recovery

You build replication with Deep Clone jobs, storage replication and CI/CD, and you write and run the failover scripts.

> **Data:** Deep Clone jobs on a schedule, plus Azure Storage replication for non-Delta files.

> **Code and config:** Terraform and Asset Bundles co-deploy to both regions on every release.

> **Failover:** your own scripts to pause the primary, start the secondary and change URLs.

> **Needed for:** active-active, cross-cloud DR, anything managed DR doesn't replicate, or fine control of the pipeline.

> **Cost:** no add-on, but engineering time to build, maintain and test the tooling.

> **Risk:** gaps appear silently when new object types are added and the scripts aren't updated.

NOT REPLICATED BY MANAGED DR · COVER THESE YOURSELF

Most teams combine the two: managed DR for catalogs and workspace assets, code and DIY jobs for the rest.

- Lakeflow pipelines
- Materialized views
- Streaming tables
- Structured Streaming outside pipelines
- Secrets
- ML models
- Model serving endpoints
- Vector search indexes
- Delta shares
- Managed volume data
- Published dashboards
- External locations and storage credentials

05 Disaster recovery runbooks

Steps shown for managed DR, with DIY notes.

A runbook is the written, step-by-step procedure someone follows under pressure at 3 a.m. Each step names who does it and how they know it worked. A runbook that has never been rehearsed is a guess.

Unplanned failover Primary region is down

	ACTION	OWNER	DONE WHEN
1	**Declare the incident and open the bridge** Same for DIY.	Incident lead	Azure Service Health confirms the outage; expected wait exceeds RTO.
2	**Confirm the secondary region is healthy** Same for DIY.	Platform admin	Service Health and a test sign-in to the secondary workspace succeed.
3	**Check the replication point** DIY: check the last successful Deep Clone and sync job times.	Platform admin	Data-loss window is known and within RPO, or escalated.
4	**Fail over: Resilience → Failover groups → Fail over** DIY: run the failover script; update DNS or client URLs.	Platform admin	Failover group leaves FAILING_OVER; stable URL points to secondary.
5	**Start compute and resume job schedules** DIY: set job concurrency above zero; raise pool minimums.	Data engineering	Clusters and SQL warehouses running; critical jobs succeed.
6	**Restore what isn't replicated** Deploy pipelines, secrets and models from code; restart streams from replicated checkpoints.	Data engineering	Streaming tables and pipelines running in the secondary.
7	**Repoint and announce** Clients not on the stable URL: update JDBC/ODBC, BI and schedulers.	Incident lead	Owners of downstream systems confirm they are connected.

Note: In an unplanned failover, data written after the last replication point can be lost. Record the gap and reconcile it after failback.

Planned DR test Quarterly rehearsal

	ACTION	OWNER	DONE WHEN
1	**Announce the test window** Same for DIY.	Incident lead	Users and downstream owners acknowledged.
2	**Stop compute in the primary** Same for DIY.	Platform admin	All clusters terminated and SQL warehouses stopped.
3	**Wait for replication to catch up** DIY: run a final Deep Clone and sync job.	Platform admin	Replication point within seconds of when writes stopped.
4	**Fail over to the secondary** DIY: run the failover script.	Platform admin	Stable URL resolves to the secondary; start time recorded.
5	**Run validation** Same for DIY.	Data engineering	Row counts, critical jobs and dashboards match expected results.
6	**Fail back and record results** Same for DIY.	Incident lead	Measured RTO and RPO logged; fixes raised as tickets.

Note: Record the measured RTO and RPO after each test and compare them with the targets. Every gap becomes a task.

Failback Return to the primary

	ACTION	OWNER	DONE WHEN
1	**Confirm the primary region is restored** Same for DIY.	Incident lead	Service Health shows resolved; test sign-in to the primary works.
2	**Schedule a maintenance window** Same for DIY.	Incident lead	Users notified of the switch-back time.
3	**Stop compute in the secondary** Same for DIY.	Platform admin	No jobs writing in the secondary.
4	**Let reversed replication catch up** DIY: sync new data and changed assets back to the primary.	Platform admin	Replication point current; audit logs show no missing writes.
5	**Fail over back to the original region** DIY: run the failback script; restore URLs.	Platform admin	Stable URL points at the primary.
6	**Start compute, resume schedules, repoint** Same for DIY.	Data engineering	Production running in the primary; DR state is ACTIVE again.

Note: After a managed DR failover, replication already runs in reverse, so failback is the same procedure with the regions swapped.

06 Deploying the secondary region as code

If the secondary region was built by hand, it has drifted from the primary by the time you need it. Define both regions from the same code, with only region-specific values changing, and every pipeline run keeps them identical.

ALWAYS DEPLOYED

Built before any incident

- VNet, subnets, private endpoints, firewall rules
- Workspace, metastore, Access Connector
- Storage accounts, external locations, storage credentials
- Key Vault, secret scopes, cluster policies, group assignments

DEPLOYED, IDLE

Present but not running

- Jobs with schedules paused (or concurrency zero)
- Pools with zero idle instances
- Clusters terminated, SQL warehouses stopped
- Dashboards and alerts pointing at secondary catalogs

AT FAILOVER

Switched on by the runbook

- Unpause schedules, start compute
- Restart streams from replicated checkpoints
- Point ingestion at secondary storage
- Repoint clients not using the stable URL

TERRAFORM · ONE MODULE, TWO REGIONS

Azure infrastructure and Unity Catalog objects for each region.

```
module "platform_primary" {
  source   = "./modules/databricks-platform"
  region   = "westeurope"
  prefix   = "prod-weu"
  is_dr    = false
}

module "platform_secondary" {
  source   = "./modules/databricks-platform"
  region   = "northeurope"
  prefix   = "prod-neu"
  is_dr    = true   # pools min_idle = 0
}
```

ASSET BUNDLE · SAME JOBS, PAUSED IN DR

Jobs and pipelines deployed to both workspaces on every release.

```
bundle:
  name: sales-pipelines

targets:
  prod:
    workspace:
      host: https://adb-111.azuredatabricks.net
  dr:
    workspace:
      host: https://adb-222.azuredatabricks.net
    resources:
      jobs:
        nightly_sales:
          schedule:
            pause_status: PAUSED
```

THE PIPELINE

1	2	3	4	5
Merge to main Reviewed change	**terraform apply** Both regions	**bundle deploy -t prod** Primary workspace	**bundle deploy -t dr** Secondary, paused	**Drift check** Plan shows no changes

Set RTO and RPO per workload

Not everything needs active-active. Agree targets with the business, then pick the cheapest strategy that meets them.

Back up as well as replicate

DR protects against losing a region. Delta history, Deep Clone backups and soft delete protect against losing data.

Rehearse every quarter

Run a planned failover, measure the real RTO and RPO, and fix what the test finds. The runbook and code change with every release.